

Lab Worksheet R04 — Lambda Functions, Tuples & Lists

Use: "Recitation-style Lab B" (40 min) of a 3-hour session. **Ground rule:** the TA never gives away answers — the TA only asks tracing questions.

Warm-up (about 5 min)

Original Problem 1: *lambda functions*. Fill in the blanks, then predict the two printed values before running: _____ / _____

```
# a) cube root of a given number
f1 = lambda x: x ** (____)      # (1) the exponent that gives a cube root

# b) product of two arguments
f2 = lambda x, y: ____ * ____   # (2)(3) the two parameter names

print(f1(8))
print(f2(4, 5))
```

Core exercises (about 30 min)

Exercise 1. Count the 1s in a tuple (original Problem 2)

(a) **Worked example** — read the original solution, then run the test.

```
def count_number_one(tup):
    count = 0
    for elem in tup:
        if elem == 1:
            count += 1
    return count

print(count_number_one((1,2,3,4,5,1,1)))
```

(b) **One-line change** — change the test tuple to `(1,2,3,4,5)`. Predict the output before running: _____

(c) **Fill in the blank** — complete the function.

```
count = 0
for elem in ____:
    if ____ == 1:
        count += ____
```

[TA tracing questions]

1. "Halfway through the test tuple — after `elem` has been 5 — what is in `count`?"
2. "What is the difference between the tuple `(1,)` and the integer `(1)`?"

Exercise 2. Common elements of two tuples (original Problem 3)

(a) **Worked example** — read the original solution, then run the test.

```
def common_elements(tup1, tup2):
    output_tuple = ()
    for x in tup1:
        for y in tup2:
            if x == y:
                output_tuple += (x,)
    return output_tuple

print(common_elements((2,3,4), (3,4,5,6)))
```

(b) One-line change — change the test to `common_elements((2,3,4), (5,6,7))`. Predict the output before running: _____

(c) Fill in the blank — complete the nested loop.

```
for x in tup1:
    for ____ in tup2:
        if x == y:
            output_tuple += (____,)
    return ____
```

[TA tracing questions]

1. "When `x` is 3 for the first time, how many times does the inner loop run — and what is in `output_tuple` when it finishes?"
2. "Why `(x,)` with a comma instead of `(x)` when growing the tuple?"

Exercise 3. Keep only sublists inside a range (original Problem 4)

(a) Worked example — remove sublists containing any element outside a given range.

```
def remove_list_range(input_list, low, high):
    output_list = []
    for i in input_list:
        if min(i) >= low and max(i) <= high:
            output_list.append(i)
    return output_list

print(remove_list_range([[2], [0], [1, 2, 3], [0, 1, 2, 3, 6, 7], [9, 11], [13, 14, 15, 17]], 13, 17))
```

(b) One-line change — change the call's range to `(0, 7)`. Predict which sublists survive before running: _____

(c) Fill in the blank — complete the filter condition.

```
for i in input_list:
    if ____ (i) >= low and ____ (i) <= high:
        output_list.____ (i)
```

[TA tracing questions]

1. "When `i` is `[9, 11]` with range 13–17, what do `min(i)` and `max(i)` return, and which side of the `and` fails?"
2. "What is in `output_list` at this point, right before the loop reaches `[13, 14, 15, 17]`?"

Self-check (about 5 min)

- ☐ I can write a one-line lambda and explain how it differs from a `def` function.
- ☐ I can trace nested loops over two tuples and predict every comparison made.
- ☐ I can explain why checking `min(i)` and `max(i)` is enough to check every element of a sublist.

Source

MIT OpenCourseWare, **6.100L Introduction to Computer Science and Programming Using Python** (Fall 2022), **Recitation 4** (`R04_problems.py` / `R04_solutions.py`). License: **CC BY-NC-SA 4.0**. Problems 1–4 of the original are used; only scaffolding was added. See <https://ocw.mit.edu> for Terms of Use.